

École Nationale Supérieure d'Informatique et
d'Analyse des Systèmes
ENSIAS

Rapport d'Avancement

DevPilot AI

Plateforme Agentic RAG pour l'Exploitation Intelligente des
Documents Privés

Réalisé par : Oussama Mahi BILAL Malih

Projet : DevPilot AI

Technologies : FastAPI, PostgreSQL, Redis, Celery, Docker, Qdrant, Ollama

Année universitaire 2025–2026

Table des matières

Résumé	4
1 Introduction générale	5
2 Vision globale du projet	6
2.1 Objectif général	6
2.2 Pipeline cible	6
3 Pourquoi utiliser ces technologies ?	8
3.1 FastAPI	8
3.2 PostgreSQL	8
3.3 Alembic	9
3.4 Redis	9
3.5 Celery	9
3.6 Docker Compose	9
3.7 Qdrant	9
3.8 Ollama	10
3.9 JWT	10
4 Architecture actuelle jusqu'à la Phase 8	11
5 Phase 1 : Initialisation du projet	13
5.1 Objectif	13
5.2 Organisation du projet	13
5.3 Organisation du backend	13
5.4 Résultat attendu	14
5.5 Vérification	14
6 Phase 2 : Backend core avec FastAPI	15
6.1 Objectif	15
6.2 Endpoint de santé	15
6.3 Configuration	15
6.4 Résultat attendu	16
6.5 Vérification	16
7 Phase 3 : Base de données avec PostgreSQL et Alembic	17
7.1 Objectif	17
7.2 Tables créées	17
7.3 Rôle des principales tables	17

7.3.1	Table users	17
7.3.2	Table workspaces	17
7.3.3	Table documents	18
7.3.4	Table chunks	18
7.4	Pourquoi utiliser Alembic ?	18
7.5	Vérification	18
8	Transition : Configuration Ollama-first	19
8.1	Pourquoi remplacer OpenAI ?	19
8.2	Configuration cible	19
8.3	Ce qui est modifié	19
8.4	Vérification	19
9	Phase 4 : Authentification JWT	21
9.1	Objectif	21
9.2	Pourquoi l'authentification est importante	21
9.3	Fonctionnement général	21
9.4	Endpoints	21
9.5	Vérification	21
10	Phase 5 : Gestion des workspaces	23
10.1	Objectif	23
10.2	Pourquoi utiliser des workspaces ?	23
10.3	Fonctionnalités	23
10.4	Endpoints	23
10.5	Vérification	24
11	Phase 6 : Upload des documents	25
11.1	Objectif	25
11.2	Types de fichiers supportés	25
11.3	Processus d'upload	25
11.4	Pourquoi valider les fichiers ?	25
11.5	Endpoints	26
11.6	Vérification	26
12	Phase 7 : Traitement asynchrone avec Celery	27
12.1	Objectif	27
12.2	Pourquoi le traitement asynchrone ?	27
12.3	Rôle de Redis	27
12.4	Cycle de vie du document	27
12.5	Traitement effectué	28
12.6	Vérification	28
13	Phase 8 : Nettoyage et découpage en chunks	29
13.1	Objectif	29
13.2	Pourquoi découper en chunks ?	29
13.3	Nettoyage du texte	29
13.4	Structure d'un chunk	30
13.5	Métadonnées	30

13.6 Vérification	30
13.7 Résultat attendu	31
14 Tests réalisés	32
14.1 Test de santé du backend	32
14.2 Test de la configuration IA	32
14.3 Test de la base de données	32
14.4 Test de Redis	32
14.5 Test de Celery	32
14.6 Test des chunks	33
15 État actuel du projet	34
16 Difficultés rencontrées	35
17 Prochaine étape : Phase 9	36
17.1 Objectif	36
17.2 Pipeline prévu	36
17.3 Objectifs détaillés	36
17.4 Importance de cette phase	36
Conclusion	38

Résumé

DevPilot AI est une plateforme intelligente destinée à l'exploitation de documents privés à travers une architecture de type *Retrieval-Augmented Generation*, appelée aussi RAG. L'objectif final du projet est de permettre à un utilisateur de téléverser ses documents, de les traiter automatiquement, puis de poser des questions en langage naturel afin d'obtenir des réponses contextualisées, vérifiables et accompagnées de sources.

Le projet ne se limite pas à un simple chatbot documentaire. Il repose sur une architecture backend complète intégrant l'authentification, la gestion des espaces de travail, l'upload documentaire, le traitement asynchrone, l'extraction de texte, le nettoyage, le découpage en chunks, puis plus tard la génération d'embeddings, la recherche vectorielle, l'orchestration d'agents, l'évaluation des réponses et l'observabilité.

À l'état actuel, le projet est arrivé à la **Phase 8**. Cela signifie que la base backend est fonctionnelle et que le pipeline d'ingestion documentaire est capable de recevoir un document, l'envoyer à un worker Celery, extraire son texte, le nettoyer, le découper en chunks, puis stocker ces chunks dans PostgreSQL.

1 Introduction générale

Les organisations modernes produisent une grande quantité de documents numériques : rapports, cahiers des charges, spécifications techniques, documents PDF, fichiers Markdown, documentations API, notes internes et décisions d’architecture. Cependant, retrouver rapidement une information fiable dans ces documents reste difficile.

Les assistants conversationnels classiques peuvent répondre à des questions générales, mais ils ne connaissent pas automatiquement les documents privés d’un utilisateur ou d’une organisation. De plus, les réponses générées ne sont pas toujours accompagnées de sources vérifiables. Pour résoudre ce problème, le paradigme RAG permet de connecter un modèle de langage à une base documentaire externe.

DevPilot AI vise donc à construire une plateforme capable de transformer des documents privés en une base de connaissances exploitable. L’utilisateur pourra, dans la version finale, poser une question et recevoir une réponse basée sur ses propres documents.

L’objectif de ce rapport est d’expliquer clairement le travail réalisé de la **Phase 1 à la Phase 8**. Le rapport ne présente pas seulement le résultat final, mais explique aussi comment chaque étape peut être reproduite sans utiliser d’outil d’intelligence artificielle.

2 Vision globale du projet

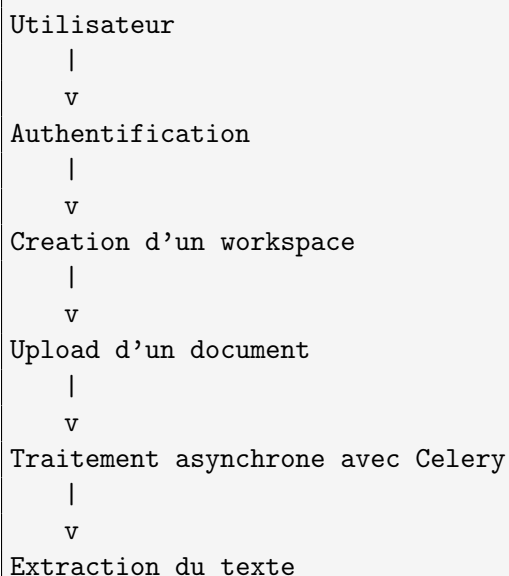
2.1 Objectif général

L'objectif général de DevPilot AI est de concevoir et développer une plateforme intelligente permettant :

- l'authentification des utilisateurs ;
- la gestion des espaces de travail ;
- l'upload de documents privés ;
- le traitement asynchrone des documents ;
- l'extraction et le nettoyage du texte ;
- le découpage du texte en chunks ;
- la génération d'embeddings ;
- la recherche sémantique dans les documents ;
- la génération de réponses contextualisées ;
- l'affichage de citations et de sources ;
- l'évaluation automatique de la qualité des réponses.

2.2 Pipeline cible

Le pipeline global visé par le projet est le suivant :



```
|  
v  
Nettoyage du texte  
|  
v  
Decoupage en chunks  
|  
v  
Generation des embeddings  
|  
v  
Stockage dans Qdrant  
|  
v  
Recherche semantique  
|  
v  
Generation d'une reponse par LLM  
|  
v  
Reponse finale avec citations
```

Les phases 1 à 8 couvrent la première partie de ce pipeline : la mise en place du backend, de la base de données, de l'authentification, des workspaces, de l'upload documentaire, du traitement asynchrone et du chunking.

3 Pourquoi utiliser ces technologies ?

3.1 FastAPI

FastAPI est utilisé comme framework backend principal. Il permet de construire rapidement des APIs modernes, performantes et bien structurées.

Il est adapté à DevPilot AI pour plusieurs raisons :

- il est rapide et basé sur Python ;
- il supporte naturellement les modèles de validation avec Pydantic ;
- il génère automatiquement une documentation Swagger ;
- il est adapté aux applications IA et data science ;
- il permet une bonne séparation entre routes, services, schémas et logique métier.

Dans ce projet, FastAPI joue le rôle de porte d'entrée principale. Il reçoit les requêtes HTTP, vérifie l'authentification, valide les données, appelle les services métier et retourne les réponses à l'utilisateur.

3.2 PostgreSQL

PostgreSQL est utilisé comme base de données relationnelle. Il stocke les données structurées du projet :

- utilisateurs ;
- workspaces ;
- documents ;
- chunks ;
- conversations ;
- messages ;
- évaluations ;
- traces d'agents ;
- usage des modèles de langage.

PostgreSQL est un bon choix parce qu'il est robuste, mature, fiable et très utilisé dans les systèmes professionnels. Il permet aussi de gérer les relations entre les entités du système.

3.3 Alembic

Alembic est utilisé pour gérer les migrations de base de données. Une migration permet de versionner la structure de la base.

Par exemple, lorsqu'on ajoute une table `documents` ou une colonne `processed_at`, Alembic permet de sauvegarder ce changement dans un fichier de migration. Cela rend le projet reproductible sur une autre machine.

Sans Alembic, il faudrait créer les tables manuellement, ce qui n'est pas professionnel et peut provoquer des erreurs.

3.4 Redis

Redis est utilisé comme broker de messages entre FastAPI et Celery.

Lorsqu'un utilisateur upload un document, FastAPI ne traite pas directement le fichier. Il envoie une tâche dans Redis. Ensuite, Celery récupère cette tâche et la traite en arrière-plan.

Redis est donc utilisé comme file d'attente légère et rapide.

3.5 Celery

Celery est utilisé pour exécuter des tâches asynchrones.

Le traitement d'un document peut prendre du temps, surtout lorsqu'il s'agit d'un PDF volumineux. Il ne faut pas bloquer la requête HTTP pendant ce traitement.

Avec Celery :

- FastAPI reçoit le document ;
- FastAPI sauvegarde les métadonnées ;
- FastAPI envoie une tâche à Redis ;
- Celery récupère la tâche ;
- Celery traite le document en arrière-plan.

Cela améliore la performance, la scalabilité et l'expérience utilisateur.

3.6 Docker Compose

Docker Compose est utilisé pour lancer tous les services du projet ensemble. Il permet d'exécuter PostgreSQL, Redis, Qdrant, FastAPI et Celery dans des conteneurs.

Cela rend le projet plus facile à installer et à reproduire. Une personne qui veut tester le projet n'a pas besoin d'installer chaque service manuellement. Elle peut simplement exécuter Docker Compose.

3.7 Qdrant

Qdrant est une base de données vectorielle. Dans les phases 1 à 8, elle est seulement préparée, mais elle sera utilisée dans la Phase 9.

Son rôle futur sera de stocker les vecteurs des chunks et de permettre la recherche sémantique. Cela signifie que l'utilisateur pourra poser une question, et le système trouvera les passages les plus proches en sens, pas seulement en mots-clés.

3.8 Ollama

Ollama est utilisé comme alternative locale et gratuite aux APIs payantes comme OpenAI.

Dans ce projet, l'objectif est d'utiliser Ollama pour :

- générer des embeddings localement ;
- générer des réponses avec un modèle local ;
- éviter les coûts liés aux APIs externes ;
- permettre des tests offline ou semi-offline.

La configuration cible est :

```
LLM_PROVIDER=ollama
EMBEDDING_PROVIDER=ollama
OLLAMA_GENERATION_MODEL=qwen3:8b
OLLAMA_EMBEDDING_MODEL=nomic-embed-text
EMBEDDING_DIMENSION=768
```

3.9 JWT

JWT, ou JSON Web Token, est utilisé pour sécuriser les routes privées.

Après connexion, l'utilisateur reçoit un token. Ce token est ensuite envoyé dans les requêtes suivantes :

```
Authorization: Bearer TOKEN
```

Le backend vérifie ce token pour identifier l'utilisateur et protéger ses données.

4 Architecture actuelle jusqu'à la Phase 8

L'architecture actuelle du projet est la suivante :

Utilisateur

|

v

FastAPI Backend

|

|----> PostgreSQL

|

|----> Redis

|

v

Celery Worker

|

v

PostgreSQL Chunks Table

Le fonctionnement est le suivant :

1. L'utilisateur s'authentifie.
2. Il crée un workspace.
3. Il upload un document.
4. FastAPI sauvegarde les métadonnées du document.
5. FastAPI envoie une tâche Celery via Redis.
6. Celery récupère la tâche.
7. Celery extrait le texte.
8. Celery nettoie le texte.
9. Celery découpe le texte en chunks.
10. Les chunks sont sauvegardés dans PostgreSQL.
11. Le document passe au statut **indexed**.

Cette architecture sépare clairement les responsabilités :

Composant	Responsabilité
FastAPI	API, validation, authentification, endpoints HTTP
PostgreSQL	Stockage des données structurées

Redis	File d'attente pour les tâches asynchrones
Celery	Traitement des documents en arrière-plan
Qdrant	Futur stockage vectoriel
Ollama	Futur fournisseur local de modèles IA

5 Phase 1 : Initialisation du projet

5.1 Objectif

La première phase consiste à créer la structure générale du projet. L'objectif n'est pas encore d'implémenter les fonctionnalités, mais de préparer une base propre et maintenable.

5.2 Organisation du projet

La structure générale choisie est :

```
devpilot-ai/  
  backend/  
  frontend/  
  monitoring/  
  docs/  
  docker-compose.yml  
  .env.example  
  README.md
```

Le dossier **backend** contient l'API et la logique métier. Le dossier **frontend** est prévu pour l'interface utilisateur. Le dossier **monitoring** servira plus tard pour Prometheus et Grafana. Le dossier **docs** contient la documentation technique.

5.3 Organisation du backend

Le backend est organisé en plusieurs modules :

```
backend/app/  
  main.py  
  core/  
  db/  
  models/  
  schemas/  
  services/  
  api/  
  workers/  
  utils/  
  agents/
```

Chaque dossier a une responsabilité précise :

Dossier	Rôle
core	Configuration, sécurité, logs, exceptions
db	Connexion à la base de données
models	Modèles SQLAlchemy liés aux tables
schemas	Schémas de validation des données
services	Logique métier
api	Routes HTTP
workers	Tâches Celery
utils	Fonctions utilitaires
agents	Futurs agents IA

5.4 Résultat attendu

À la fin de cette phase, le projet doit avoir une structure claire, même si les fonctionnalités ne sont pas encore implémentées.

5.5 Vérification

Pour vérifier la structure :

```
tree -L 4
```

Pour vérifier Docker Compose :

```
docker compose config
```

6 Phase 2 : Backend core avec FastAPI

6.1 Objectif

La deuxième phase consiste à rendre le backend fonctionnel avec FastAPI.

Les éléments mis en place sont :

- l'application FastAPI;
- un endpoint de santé;
- la configuration par variables d'environnement;
- CORS;
- les logs;
- la gestion globale des erreurs.

6.2 Endpoint de santé

Le endpoint `/health` permet de vérifier que le backend fonctionne.

```
curl http://localhost:8000/health
```

Résultat attendu :

```
{
  "status": "ok",
  "service": "DevPilot AI API"
}
```

6.3 Configuration

La configuration est gérée à travers un fichier `.env`. Cela permet d'éviter de mettre les valeurs sensibles directement dans le code.

Exemples de variables :

```
APP_NAME=DevPilot AI
APP_ENV=development
DATABASE_URL=postgresql+asyncpg://devpilot:devpilot@postgres:5432/devpilot
REDIS_URL=redis://redis:6379/0
QDRANT_URL=http://qdrant:6333
SECRET_KEY=change_this_secret_key
```


6.4 Résultat attendu

À la fin de cette phase :

- le backend démarre correctement ;
- `/health` répond avec le statut 200 ;
- les logs de démarrage apparaissent ;
- les erreurs sont retournées proprement.

6.5 Vérification

```
docker compose up -d backend
curl http://localhost:8000/health
docker compose logs backend
```

7 Phase 3 : Base de données avec PostgreSQL et Alembic

7.1 Objectif

La Phase 3 consiste à configurer PostgreSQL et Alembic, puis à créer les tables principales du projet.

7.2 Tables créées

Les tables principales sont :

- `users`
- `workspaces`
- `workspace_members`
- `documents`
- `chunks`
- `conversations`
- `messages`
- `retrieved_chunks`
- `agent_runs`
- `evaluations`
- `llm_usage`

7.3 Rôle des principales tables

7.3.1 Table `users`

Elle stocke les utilisateurs de la plateforme.

7.3.2 Table `workspaces`

Elle stocke les espaces de travail créés par les utilisateurs.

7.3.3 Table documents

Elle stocke les métadonnées des fichiers uploadés, comme le nom du fichier, le type, le statut et le chemin de stockage.

7.3.4 Table chunks

Elle stocke les fragments textuels extraits des documents. Cette table est essentielle pour la future recherche sémantique.

7.4 Pourquoi utiliser Alembic ?

Alembic permet de versionner la structure de la base de données. Lorsqu'une table ou une colonne est ajoutée, Alembic permet de générer une migration et de l'appliquer proprement.

7.5 Vérification

```
docker compose up -d postgres
docker compose exec backend alembic upgrade head
docker compose exec postgres psql -U devpilot -d devpilot -c "\dt"
```

Résultat attendu : toutes les tables doivent apparaître.

8 Transition : Configuration Ollama-first

8.1 Pourquoi remplacer OpenAI ?

Au début, le projet utilisait OpenAI comme configuration par défaut. Cependant, OpenAI nécessite une clé API et peut générer des coûts.

Pour un projet étudiant, il est préférable de préparer une approche locale et gratuite avec Ollama.

8.2 Configuration cible

La configuration cible est :

```
LLM_PROVIDER=ollama
EMBEDDING_PROVIDER=ollama
OLLAMA_BASE_URL=http://host.docker.internal:11434
OLLAMA_GENERATION_MODEL=qwen3:8b
OLLAMA_EMBEDDING_MODEL=nomic-embed-text
EMBEDDING_DIMENSION=768
```

8.3 Ce qui est modifié

Cette transition modifie :

- le fichier `.env` ;
- le fichier `.env.example` ;
- la configuration backend ;
- les logs de démarrage ;
- la documentation.

Elle ne modifie pas la base de données, car les tables restent valables quel que soit le fournisseur IA utilisé.

8.4 Vérification

```
curl http://localhost:8000/api/v1/system/ai-config
```

Résultat attendu :

```
{  
  "llm_provider": "ollama",  
  "embedding_provider": "ollama",  
  "generation_model": "qwen3:8b",  
  "embedding_model": "nomic-embed-text",  
  "embedding_dimension": 768  
}
```

9 Phase 4 : Authentication JWT

9.1 Objectif

La Phase 4 ajoute l'authentification des utilisateurs.

L'utilisateur doit pouvoir :

- s'inscrire ;
- se connecter ;
- recevoir un token JWT ;
- accéder aux routes protégées.

9.2 Pourquoi l'authentification est importante

DevPilot AI manipule des documents privés. Il est donc indispensable de garantir qu'un utilisateur ne peut accéder qu'à ses propres données.

9.3 Fonctionnement général

Le fonctionnement est le suivant :

1. L'utilisateur s'inscrit avec email, nom et mot de passe.
2. Le mot de passe est hash.
3. L'utilisateur se connecte.
4. Le backend vrifie le mot de passe.
5. Le backend gnre un JWT.
6. L'utilisateur utilise ce token pour les requetes suivantes.

9.4 Endpoints

```
POST /api/v1/auth/register  
POST /api/v1/auth/login  
GET /api/v1/auth/me
```

9.5 Vérification

Inscription :

```
curl -X POST http://localhost:8000/api/v1/auth/register \
-H "Content-Type: application/json" \
-d '{
  "email": "oussama@example.com",
  "full_name": "Oussama Mahi",
  "password": "StrongPassword123"
}'
```

Connexion :

```
curl -X POST http://localhost:8000/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{
  "email": "oussama@example.com",
  "password": "StrongPassword123"
}'
```

Route protégée :

```
curl http://localhost:8000/api/v1/auth/me \
-H "Authorization: Bearer TOKEN"
```

10 Phase 5 : Gestion des workspaces

10.1 Objectif

La Phase 5 ajoute la gestion des workspaces. Un workspace est un espace de travail privé dans lequel l'utilisateur peut organiser ses documents.

10.2 Pourquoi utiliser des workspaces ?

Les workspaces permettent de séparer les documents par projet, équipe ou contexte.
Exemple :

```
Utilisateur A
  Workspace 1
    Documents
  Workspace 2
    Documents

Utilisateur B
  Workspace 3
    Documents
```

Cette organisation prépare aussi une future architecture multi-tenant.

10.3 Fonctionnalités

Les fonctionnalités implémentées sont :

- création d'un workspace ;
- liste des workspaces de l'utilisateur ;
- récupération d'un workspace ;
- modification d'un workspace ;
- isolation entre utilisateurs.

10.4 Endpoints

```
POST /api/v1/workspaces
GET /api/v1/workspaces
GET /api/v1/workspaces/{workspace_id}
```



```
PATCH /api/v1/workspaces/{workspace_id}
DELETE /api/v1/workspaces/{workspace_id}
```

10.5 Vérification

Créer un workspace :

```
curl -X POST http://localhost:8000/api/v1/workspaces \
-H "Authorization: Bearer TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "name": "DevPilot Workspace",
  "description": "Workspace de test"
}'
```

Tester l'isolation :

- créer un utilisateur A ;
- créer un utilisateur B ;
- créer un workspace avec A ;
- essayer d'accéder à ce workspace avec B ;
- le résultat attendu est 403 Forbidden ou 404 Not Found.

11 Phase 6 : Upload des documents

11.1 Objectif

La Phase 6 permet à un utilisateur d'uploader un document dans un workspace.

11.2 Types de fichiers supportés

Les types de fichiers supportés sont :

- .txt
- .md
- .pdf

11.3 Processus d'upload

Le processus est le suivant :

1. L'utilisateur choisit un fichier.
2. Le backend vrifie le token.
3. Le backend vrifie l'accs au workspace.
4. Le backend valide le type du fichier.
5. Le backend sauvegarde le fichier.
6. Le backend cre une ligne dans la table documents.
7. Le document reçoit le statut queued.

11.4 Pourquoi valider les fichiers ?

La validation permet d'éviter :

- l'upload de fichiers dangereux ;
- l'upload de types non supportés ;
- les attaques par path traversal ;
- les fichiers trop volumineux.

11.5 Endpoints

```
POST /api/v1/workspaces/{workspace_id}/documents
GET /api/v1/workspaces/{workspace_id}/documents
GET /api/v1/documents/{document_id}
GET /api/v1/documents/{document_id}/status
DELETE /api/v1/documents/{document_id}
```

11.6 Vérification

Upload valide :

```
curl -X POST http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID/documents \
-H "Authorization: Bearer $TOKEN" \
-F "file=@sample.txt"
```

Upload invalide :

```
curl -X POST http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID/documents \
-H "Authorization: Bearer $TOKEN" \
-F "file=@virus.exe"
```

Résultat attendu pour un fichier invalide :

```
400 Bad Request
```

12 Phase 7 : Traitement asynchrone avec Celery

12.1 Objectif

La Phase 7 ajoute le traitement asynchrone des documents avec Celery et Redis.

Avant cette phase, le document était seulement uploadé. Après cette phase, il est envoyé à un worker pour être traité en arrière-plan.

12.2 Pourquoi le traitement asynchrone ?

Le traitement d'un fichier peut prendre du temps. Si FastAPI traite directement le document pendant la requête HTTP, l'utilisateur doit attendre.

Avec Celery :

- FastAPI répond rapidement ;
- le traitement se fait en arrière-plan ;
- le système devient plus scalable ;
- les erreurs peuvent être gérées séparément.

12.3 Rôle de Redis

Redis reçoit les tâches envoyées par FastAPI. Celery récupère ensuite ces tâches depuis Redis.

12.4 Cycle de vie du document

Le document passe par les statuts suivants :

```
queued -> processing -> indexed
```

En cas d'erreur :

```
queued -> processing -> failed
```

12.5 Traitement effectué

Dans cette phase, Celery :

- récupère le document ;
- lit le fichier ;
- extrait le texte ;
- met à jour le statut ;
- marque le document comme `indexed` ou `failed`.

12.6 Vérification

Vérifier Redis :

```
docker compose exec redis redis-cli ping
```

Résultat attendu :

```
PONG
```

Observer Celery :

```
docker compose logs -f celery_worker
```

Vérifier le statut :

```
curl http://localhost:8000/api/v1/documents/$DOCUMENT_ID/status \
-H "Authorization: Bearer $TOKEN"
```

Vérifier en base :

```
docker compose exec postgres psql -U devpilot -d devpilot -c \
"select id, filename, status, processed_at from documents order by created_at
desc limit 5;"
```

13 Phase 8 : Nettoyage et découpage en chunks

13.1 Objectif

La Phase 8 transforme le texte brut extrait des documents en chunks structurés. Cette étape est essentielle pour préparer le futur pipeline RAG.

13.2 Pourquoi découper en chunks ?

Un document peut être long. Il n'est pas efficace d'envoyer tout le document à un modèle de langage ou à un moteur de recherche vectorielle.

Le chunking permet de :

- diviser un document en petites unités ;
- faciliter la recherche sémantique ;
- retrouver uniquement les passages pertinents ;
- améliorer les citations ;
- réduire la taille du contexte envoyé au modèle.

13.3 Nettoyage du texte

Le nettoyage consiste à rendre le texte plus propre avant le découpage.

Il peut inclure :

- la normalisation des espaces ;
- la suppression des lignes vides excessives ;
- la conservation des titres ;
- la conservation des blocs importants ;
- la préparation du texte pour le chunking.

Exemple avant nettoyage :

```
DevPilot AI has many spaces.
```

```
This text contains bad whitespace.
```

Exemple après nettoyage :

```
DevPilot AI has many spaces.  
  
This text contains bad whitespace.
```

13.4 Structure d'un chunk

Chaque chunk contient :

- document_id
- workspace_id
- content
- chunk_index
- token_count
- metadata
- created_at

La table chunks contient actuellement :

```
document_id uuid  
workspace_id uuid  
content text  
chunk_index integer  
token_count integer  
metadata jsonb  
vector_id varchar  
created_at timestamp with time zone  
id uuid
```

Le champ `vector_id` est prévu pour la Phase 9. Il permettra de relier chaque chunk à son vecteur stocké dans Qdrant.

13.5 Métadonnées

La colonne `metadata` permet de stocker des informations supplémentaires sur le chunk. Exemple :

```
{  
  "start_char": 0,  
  "end_char": 800  
}
```

Ces informations sont utiles pour la traçabilité, le debug et les futures citations.

13.6 Vérification

Vérifier le document :

```
docker compose exec postgres psql -U devpilot -d devpilot -c \  
"select id, filename, status, processed_at from documents where id = '  
  $DOCUMENT_ID';"
```

Vérifier les chunks :

```
docker compose exec postgres psql -U devpilot -d devpilot -c \  
"select chunk_index, token_count, left(content, 120) as preview from chunks  
  where document_id = '$DOCUMENT_ID' order by chunk_index;"
```

Vérifier les métadonnées :

```
docker compose exec postgres psql -U devpilot -d devpilot -c \  
"select chunk_index, metadata from chunks where document_id = '$DOCUMENT_ID'  
  order by chunk_index limit 5;"
```

13.7 Résultat attendu

À la fin de la Phase 8 :

- les documents valides passent au statut `indexed`;
- les fichiers corrompus passent au statut `failed`;
- les chunks sont créés dans PostgreSQL;
- chaque chunk contient un contenu non vide;
- chaque chunk possède un `chunk_index`;
- chaque chunk possède un `token_count`;
- chaque chunk possède des métadonnées.

14 Tests réalisés

14.1 Test de santé du backend

```
curl http://localhost:8000/health
```

Résultat attendu :

```
{  
  "status": "ok"  
}
```

14.2 Test de la configuration IA

```
curl http://localhost:8000/api/v1/system/ai-config
```

Résultat attendu :

```
llm_provider = ollama  
embedding_provider = ollama  
generation_model = qwen3:8b  
embedding_model = nomic-embed-text
```

14.3 Test de la base de données

```
docker compose exec postgres psql -U devpilot -d devpilot -c "\dt"
```

14.4 Test de Redis

```
docker compose exec redis redis-cli ping
```

Résultat attendu :

```
PONG
```

14.5 Test de Celery

```
docker compose logs -f celery_worker
```

14.6 Test des chunks

```
docker compose exec postgres psql -U devpilot -d devpilot -c \  
"select chunk_index, token_count, left(content, 120) as preview from chunks  
order by created_at desc limit 10;"
```

15 État actuel du projet

Module	État
Structure du projet	Terminé
Docker Compose	Fonctionnel
Backend FastAPI	Fonctionnel
Configuration environnement	Fonctionnelle
PostgreSQL	Fonctionnel
Alembic	Fonctionnel
Authentification JWT	Fonctionnelle
Workspaces	Fonctionnels
Isolation des données	Fonctionnelle
Upload documents	Fonctionnel
Redis	Fonctionnel
Celery Worker	Fonctionnel
Extraction texte	Fonctionnelle
Nettoyage texte	Fonctionnel
Chunking	Fonctionnel
Stockage chunks PostgreSQL	Fonctionnel
Gestion erreurs fichiers corrompus	Fonctionnelle
Ollama config	Préparée
Qdrant	Préparé
Embeddings	Prochaine phase
Recherche sémantique	Prochaine phase
Chat RAG	Prochaine phase

L'avancement actuel du MVP est estimé entre **35% et 40%**. La base backend et le pipeline d'ingestion documentaire sont déjà fonctionnels.

16 Difficultés rencontrées

Plusieurs difficultés techniques ont été rencontrées durant les phases 1 à 8 :

- configuration correcte de Docker Compose ;
- communication entre FastAPI, PostgreSQL, Redis et Celery ;
- partage des fichiers uploadés entre le backend et le worker Celery ;
- gestion des migrations Alembic ;
- mise en place de l'authentification JWT ;
- isolation des workspaces entre utilisateurs ;
- validation des fichiers uploadés ;
- gestion des statuts des documents ;
- traitement des fichiers corrompus ;
- remplacement de la configuration OpenAI par une approche Ollama-first ;
- vérification manuelle des chunks dans PostgreSQL.

Ces difficultés ont permis d'améliorer la robustesse de l'architecture et de mieux comprendre le fonctionnement d'un pipeline backend professionnel.

17 Prochaine étape : Phase 9

17.1 Objectif

La Phase 9 consistera à transformer les chunks textuels en vecteurs numériques appelés embeddings.

17.2 Pipeline prévu

```
Chunks PostgreSQL
  |
  v
Generation des embeddings avec Ollama
  |
  v
Stockage des vecteurs dans Qdrant
  |
  v
Recherche sémantique
```

17.3 Objectifs détaillés

Les objectifs de la Phase 9 sont :

- générer les embeddings avec Ollama ;
- créer une collection dans Qdrant ;
- stocker chaque chunk sous forme vectorielle ;
- sauvegarder le `vector_id` dans PostgreSQL ;
- tester la recherche vectorielle ;
- préparer le futur chat RAG.

17.4 Importance de cette phase

Actuellement, les chunks sont stockés comme texte. Après la Phase 9, ils seront aussi représentés sous forme de vecteurs. Cela permettra de rechercher des passages par similarité sémantique.

Par exemple, si l'utilisateur demande :

What is the architecture of the system?

Le système pourra retrouver des chunks parlant de FastAPI, PostgreSQL, Redis, Celery et Qdrant, même si la question ne contient pas exactement tous ces mots.

Conclusion

À ce stade, DevPilot AI possède une base backend solide et professionnelle. Les phases 1 à 8 ont permis de construire l'infrastructure principale du projet : FastAPI, PostgreSQL, Alembic, JWT, workspaces, upload documentaire, Redis, Celery, extraction de texte, nettoyage et chunking.

La Phase 8 représente une étape clé, car elle transforme les documents bruts en unités exploitables pour la recherche sémantique. Les chunks créés dans PostgreSQL serviront directement à la génération d'embeddings dans la Phase 9.

Le projet avance progressivement vers son objectif principal : construire une plateforme intelligente capable d'exploiter des documents privés et de fournir des réponses fiables, contextualisées et vérifiables.

État actuel : Phase 8 validée.

Prochaine étape : embeddings avec Ollama et stockage vectoriel dans Qdrant.